

Rôle de S et α : théorie et validation (XOR, Iris)

1 Notations

$x \in \{0, 1\}$: feature ; $y \in \{0, 1\}$: label. $v, \bar{v}$: automates des littéraux x et $\neg x$. $S, \alpha \in (0, 1]$: paramètres de la règle à 4 cas (Section 2 de *Démonstration de convergence*).
Statistiques du canal de bruit : $c = P(x=1)$, $a = P(y=1 \mid x=1)$, $b = P(y=1 \mid x=0)$.
 $\rho = p^+/p^-$: $\rho > 1 \Rightarrow$ l'automate converge vers Include; $\rho < 1 \Rightarrow$ vers Exclude (Lemme 6.4 du même document).

2 Formule du seuil critique

Pour le littéral pertinent (cas IDENTITY, $a > b$), le document précédent établit :

$$p_v^+ = (1 - c)(1 - b) S, \quad p_v^- = (1 - c) b \alpha + c(1 - a) S.$$

$\rho_v > 1 \iff p_v^+ > p_v^-$. En résolvant pour α à S fixé :

$$\alpha^*(S) = \frac{\Delta S}{(1 - c) b}, \quad \Delta = (1 - c)(1 - b) - c(1 - a).$$

Pour $\alpha < \alpha^*(S)$: $\rho_v > 1$, le littéral apprend correctement. Pour $\alpha > \alpha^*(S)$: $\rho_v < 1$, il apprend dans le mauvais sens.

3 Mesure de c , a , b sur XOR

Sur `NoisyXORTrainingData.txt` (12 features, 5000 exemples) :

1. Chaque feature seule : $P(x_i=1) \approx 0,5$ et $P(x_i=y) \approx 0,5$ — aucune ne prédit y seule (attendu pour un XOR).
2. Chaque paire (i, j) : $P(x_i \oplus x_j = y)$. Une seule paire sort du lot, $(1, 2)$, à 60,3 % — toutes les autres ≈ 50 %.
3. Donc bruit = $1 - 0,603 = 0,397$. En conditionnant sur x_1 , x_2 devient le littéral pertinent : $c = 0,5$, $a = 0,603$, $b = 0,397$.

D'où $\Delta = 0,5 \cdot 0,603 - 0,5 \cdot 0,397 = 0,103$ et

$$\alpha^*(S) = \frac{0,103 S}{0,5 \times 0,397} = 0,519 S.$$

4 Validation empirique

Balayage $S, \alpha \in \{0,1; 0,3; 0,35; 0,5; 0,7; 0,9; 1,0\}$, $M = 200$, $N = 50$, total = 1500, 20 runs (`random_stump_perclause_boost.cpp`).

$S \backslash \alpha$	0,1	0,3	0,35	0,5	0,7	0,9	1,0
0,1	78,1	48,9	48,9	48,9	48,9	48,9	48,9
0,3	86,1	74,2	64,3	48,9	48,9	48,9	48,9
0,5	85,4	91,5	82,7	61,6	48,9	48,9	48,9
0,7	84,1	95,8	89,0	77,0	51,4	48,9	48,9
0,9	81,9	97,5	95,8	80,1	69,4	48,9	48,9
1,0	81,1	98,6	99,1	83,4	75,8	56,5	48,9

Table 1: Accuracy (% , 20 runs). Fond rouge : 48,9 %, variance nulle — effondrement total vers un classifieur trivial.

S	α^* prédit	Dernier α non-effondré (empirique)
0,1	0,052	0,1 (78,1 %, déjà dégradé)
0,3	0,156	0,3 (74,2 %)
0,5	0,259	0,35 (82,7 %)
0,7	0,363	0,35 (89,0 %)
0,9	0,467	0,5 (80,1 %)
1,0	0,519	0,5 (83,4 %)

Table 2: Seuil prédit vs position réelle de l’effondrement. Le dernier α non-effondré est toujours juste sous $\alpha^*(S)$, l’effondrement (48,9 %) commence toujours juste au-dessus.

5 Conclusion

Le meilleur point trouvé ($S=1,0, \alpha=0,35 \rightarrow 99,1\%$) est celui qui maximise la marge sous $\alpha^*(S) = 0,519$ sans réduire α au point de ralentir la correction du bruit ($\alpha = 0,1$ plafonne à 78–86 % quel que soit S , trop faible pour purger vite les mauvais états initiaux). La formule $\alpha^*(S)$ prédit correctement, sur les 42 cellules testées, où l’effondrement total commence.

6 Étude de cas : Iris

6.1 Différence de départ avec XOR

Sur XOR, aucune feature seule ne prédit y ($a \approx b$ pour toutes) : il faut en combiner deux. Sur Iris, plusieurs features prédisent y **seules**, avec un écart $a - b$ très supérieur à celui de XOR (0,206).

6.2 Mesure de c, a, b sur Iris (BinaryIrisData.txt, 16 features, 149 exemples)

Pour chaque feature et chaque classe (one-vs-rest), on mesure $c = P(x=1)$, $a = P(y=1 \mid x=1)$, $b = P(y=1 \mid x=0)$ et on calcule $\alpha^*(S=1)$ avec la même formule que pour XOR :

Feature	Classe	c	a	b	$\alpha^*(S=1)$
8	1	0,631	0,468	0,091	≈ 0 (fragile)
11	2	0,624	0,484	0,089	0,60 (fragile)
2	2	0,282	0,738	0,178	4,05
7	0	0,369	0,673	0,138	4,85
11	1	0,624	0,516	0,018	10,00
16	2	0,349	0,904	0,031	29,67 (quasi incassable)

Table 3: 7 des 16 features sont constantes (toujours la même valeur, aucun signal possible). Parmi les 9 restantes, la force du signal varie de très fragile ($\alpha^* \approx 0$) à quasi incassable ($\alpha^* \approx 30$).

6.3 Validation empirique

Balayage $S, \alpha \in \{0,05; 0,1; 0,3; 0,5; 0,7; 1,0\}$, $M = 1500$, $K = 2$, $N = 100$, `total = 20 000`, 20 runs (`random_stump_multifeature_multiclass.cpp`).

$S \backslash \alpha$	0,05	0,1	0,3	0,5	0,7	1,0
0,05	95,5	92,0	89,8	88,0	88,0	88,2
0,1	95,5	95,7	90,7	90,7	89,3	88,0
0,3	95,5	95,5	95,3	90,8	92,0	91,8
0,5	95,5	95,5	95,5	94,8	92,8	92,2
0,7	95,5	95,5	95,5	95,5	95,2	92,8
1,0	95,5	95,5	95,5	95,5	95,5	95,7

Table 4: Accuracy (% , 20 runs). Contrairement à XOR : jamais d’effondrement total (le pire reste à 88 %, largement au-dessus du hasard à 33 %).

6.4 Conclusion

Deux différences avec XOR expliquent le tableau :

- La plupart des features/classes ont $\alpha^* \gg 1$: hors de la plage valide $(0, 1]$, la destination ne bascule jamais — d’où la zone plate à 95,5 % dès que S est assez grand.
- La dégradation à petit S /grand α (95,5 % $\rightarrow$ 88 %) n’est donc pas un basculement de destination : c’est un effet de *vitesse* (convergence pas assez rapide dans les 20 000 tirages), comme la colonne $\alpha = 0,1$ de XOR (Section 5). Les quelques features fragiles ($\alpha^* \approx 0,6$, dans la plage testée) existent, mais leur effet se dilue dans le vote de 1500 clauses tirées parmi 120 paires de features possibles.

XOR est proche du seuil critique (bascule possible et observée) ; Iris en est loin (jamais de bascule, seulement une question de vitesse).

7 Étude de cas : Digits

7.1 Mesure de c, a, b sur Digits (BinaryDigitsTrainingData192.txt, 192 features, 1437 exemples, 10 classes)

Sur $192 \times 10 = 1920$ combinaisons feature/classe, 312 ont un signal IDENTITY exploitable ($|a-b| \geq 0,02$). Contrairement à Iris, plusieurs sont **très fragiles** — $\alpha^*(S=1)$ tombe sous 1 pour un nombre significatif de features, pas seulement 1 ou 2 cas isolés comme sur Iris :

Feature	Classe	c	a	b	$\alpha^*(S=1)$
55	0	0,511	0,125	0,075	0,113 (très fragile)
26	6	0,505	0,121	0,081	0,293
47	9	0,522	0,143	0,048	0,333
55	9	0,511	0,129	0,064	0,378
18	9	0,502	0,121	0,074	0,547
180	1	0,500	0,132	0,082	0,593
34	4	0,410	0,219	0,007	63,67
34	0	0,410	0,241	0,004	132,67
31	0	0,390	0,255	0,002	229,00

Table 5: Échantillon des cas les plus fragiles (haut) et les plus robustes (bas) parmi les 312 combinaisons exploitables. Contrairement à Iris (une seule feature fragile trouvée, $\alpha^*=0,60$), Digits en a plusieurs sous 1.

Prédiction avant validation (voir Section 7.3 pour le résultat réel, qui infirme cette prédiction) : plus de features fragiles dans la plage testable $\Rightarrow$ on s’attend à une dégradation plus marquée que sur Iris à petit S /grand α , potentiellement plus proche du comportement de XOR.

7.2 Validation empirique

Balayage $S, \alpha \in \{0,05; 0,1; 0,3; 0,5; 0,7; 1,0\}$, $M = 1000$, $K = 2$, $N = 100$, `total = 20 000`, 2 runs (`random_stump_multifeature_multiclass.cpp`).

7.3 Conclusion — la prédiction était fausse, et pourquoi

La prédiction de la section précédente ne se vérifie pas : Digits est en réalité **plus stable** qu’Iris, pas moins, malgré des features individuellement plus fragiles ($\alpha^* = 0,113$ contre 0,60 pour le pire cas d’Iris).

L’erreur venait d’un raisonnement incomplet : la fragilité d’une feature *seule* ne suffit pas à prédire l’effet sur le vote global — il faut aussi tenir compte du nombre de combinaisons possibles. Digits a 192 features, donc $\binom{192}{2} \approx 18\,300$ paires conditionnables, contre $\binom{16}{2} = 120$ pour Iris (environ $150\times$ plus). Même si quelques features sont fragiles, la probabilité qu’une clause tire précisément une combinaison qui isole l’une d’elles

$S \backslash \alpha$	0,05	0,1	0,3	0,5	0,7	1,0
0,05	95,6	95,8	96,5	96,4	96,4	97,1
0,1	95,6	95,6	95,8	96,5	96,9	96,5
0,3	95,6	95,6	95,4	96,0	96,0	96,8
0,5	95,6	95,6	95,4	95,6	95,0	96,3
0,7	95,6	95,6	95,4	95,6	95,3	95,0
1,0	95,6	95,6	95,4	95,6	95,6	94,2

Table 6: Accuracy (% , 2 runs). Aucun effondrement nulle part : tout reste entre 94,2 % et 97,1 %, plus stable qu’Iris (qui descendait jusqu’à 88 %).

comme littéral résiduel est bien plus faible sur Digits — l’effet se dilue davantage dans le vote de milliers de clauses, malgré une fragilité individuelle plus forte.

Note 7.1 (Leçon méthodologique, corrigée après le test MNIST — voir Section 8). Le nombre de combinaisons possibles seul ne suffit pas à prédire la dilution : MNIST (Section 8) a encore plus de combinaisons que Digits, et pourtant montre *beaucoup plus* de sensibilité à S, α . Le vrai facteur est le **ratio entre M (nombre de clauses) et le nombre de combinaisons possibles $\binom{n}{K}$** — pas la taille de l’espace seule. Voir Section 8.3.

8 Étude de cas : MNIST

8.1 Mesure de c, a, b sur MNIST (784 features, 60 000 exemples, 10 classes)

Sur $784 \times 10 = 7840$ combinaisons feature/classe, 1905 ont un signal IDENTITY exploitable. Le pire cas trouvé est encore plus fragile que sur Digits : $\alpha^*(S=1) = 0,051$ (feature 434, classe 1), contre 0,113 pour Digits.

8.2 Validation empirique

Balayage $S, \alpha \in \{0,05; 0,1; 0,3; 0,5; 0,7; 1,0\}$, $M = 500$ (réduit pour un balayage rapide), $K = 2$, $N = 100$, total = 20 000, 1 run (mnist_onyxia).

8.3 Pourquoi MNIST est sensible alors que Digits ne l’est pas : le ratio M /combinaisons

MNIST a *plus* de combinaisons possibles que Digits ($\binom{784}{2} \approx 307\,000$ contre 18 300), donc la dilution « espace combinatoire seul » prédirait moins de sensibilité, pas plus. Le facteur manquant : M testé ici est **deux fois plus petit** pour MNIST (500) que pour Digits (1000). Rapporté à la taille de l’espace :

$$\text{Digits : } \frac{1000}{18\,300} \approx 5,5 \%, \quad \text{MNIST : } \frac{500}{307\,000} \approx 0,16 \% \quad (34 \times \text{plus faible}).$$

$S \setminus \alpha$	0,05	0,1	0,3	0,5	0,7	1,0
0,05	85,9	80,5	58,4	52,7	51,0	39,6
0,1	85,9	85,9	71,5	58,1	51,7	48,8
0,3	85,9	85,9	85,9	77,4	71,7	63,7
0,5	85,9	85,9	85,9	85,9	79,7	74,2
0,7	85,9	85,9	85,9	85,9	85,9	78,8
1,0	85,9	85,9	85,9	85,9	85,9	85,5

Table 7: Accuracy (% , 1 run). Forte dégradation (85,9 % $\rightarrow$ 39,6 %) quand α dépasse S — contrairement à Digits, quasi plat.

Avec un tirage aussi clairsemé, chaque clause pèse proportionnellement plus dans le vote : les clauses qui tombent sur une combinaison fragile *et* un mauvais réglage ne sont plus diluées par la masse des autres. C’est cette rareté relative de l’échantillonnage, pas la taille absolue de l’espace, qui gouverne la sensibilité à S, α .

9 Synthèse

Dataset	$\binom{n}{K}$	M testé	$M / \binom{n}{K}$	Sensibilité
XOR	12	200	17 %	Effondrement net (mur à α^*)
Iris	120	1500	1250 %	Dégradation modérée (88–95,5 %)
Digits	18 300	1000	5,5 %	Quasi aucune (94,2–97,1 %)
MNIST	307 000	500	0,16 %	Forte (39,6–85,9 %)

La colonne qui explique vraiment la sensibilité est $M / \binom{n}{K}$ (le « taux d’échantillonnage » de l’espace combinatoire), pas $\binom{n}{K}$ seul : Iris (ratio le plus haut) est robuste, MNIST (ratio le plus bas, malgré le plus grand espace) est le plus sensible. Un M plus grand sur MNIST (non testé ici, piste à suivre) devrait réduire cette sensibilité en rapprochant le ratio de celui de Digits.